Tools Ecosystem

Day 1 - Introduction and Fundamentals

Tiago Sousa

What We'll Cover

By the end of this session, you will be able to:

1. **Map** the AI coding tools landscape on a spectrum of autonomy
2. **Compare** interaction models - inline, IDE agent, CLI agent
3. **Evaluate** trade-offs - autonomy, IDE lock-in, model flexibility, cost
4. **Connect** tools to what you'll use in this course

The Spectrum of Autonomy

> *Every AI coding tool sits somewhere on a line from **quiet assistant** to **autonomous agent**.*

Understanding where a tool sits - and *why* - is the key to choosing the right tool for the right job.

LEVELS	SELF-DRIVING CARS	AI DEV AGENTS
00	No automation: human does everything	No AI assistance: humans write code, tests, docs
01	Driver assistance (cruise control, lane keeping)	Coding assistant (suggestions, explanations, refactoring)
02	Partial automation (steering + speed; hands-off but eyes-on)	Partial autonomy (features, tests, multi-file changes with oversight)
03	Conditional automation (autonomous in defined conditions)	Conditional autonomy (autonomy within well-tested, spec-defined scope with good tests)
04	High automation (most scenarios; rare human takeover)	High autonomy (handles most dev tasks; human reviews exceptions)
05	Full automation (all conditions; no human needed)	Full autonomy (from business requirements to deployment & maintenance)

GitHub Copilot

The AI Assistant

Copilot - What It Is

- First widely adopted AI pair programmer (GitHub/Microsoft, 2021)
- Lives **inside your existing editor** - VS Code, JetBrains, Neovim
- Think: a very smart autocomplete that understands *intent*, not just syntax

Two interaction modes:

1. **Inline suggestions** - ghost text as you type, Tab to accept
2. **Chat panel** - conversational sidebar for explanations, generation, debugging

Copilot - Inline Suggestions

The signature experience:

- Low friction
- Always present
- Never intrusive
- Press **Tab** to accept

```typescript
// create a class in TypeScript to represent a student that has a name, an id, and a list of courses
// the class should have method to add and remove a course to the list of courses

class Student {
  name: string;
  id: number;
  courses: string[];

  constructor(name: string, id: number, courses: string[]) {
    this.name = name;
    this.id = id;
    this.courses = courses;
  }

  addCourse(course: string) {
    this.courses.push(course);
  }

  removeCourse(course: string) {
    this.courses = this.courses.filter((c) => c !== course);
  }
}
```

Copilot - Chat Panel

Ask Copilot to:

- Explain code
- Generate functions
- Debug errors

Like ChatGPT, but with your codebase as context.

File Edit Selection View Go ···

xxx

CHAT

PROJECT REVIEW REQUEST

Code Review: Task Manager Application

Overall Assessment

This is a well-structured, functional task manager application. The codebase demonstrates good fundamentals but has several areas that warrant attention.

HTML Review (index.html)

Strengths

- Good use of semantic HTML5 elements (`<header>`, `<main>` is missing though)
- Proper `lang` attribute and meta tags
- Clean structure with meaningful class names

Issues to Address

Missing Semantic Structure
The container wrapping the content should use `<main>` element per project guidelines. This improves screen reader navigation.

Accessibility Gaps

- The task input lacks an associated `<label>` element. Screen readers won't properly identify the field's purpose
- Filter buttons lack `aria-pressed` state to communicate their toggle status
- The empty state announcement won't be communicated to screen readers when it changes

app.js

Review code for quality and adherence to best practices.

Reviewer Claude Opus 4.5

Open Chat Ctrl + Alt + I
Show All Commands Ctrl + Shift + P
Go to File Ctrl + P

Copilot - Strengths & Limitations

Strengths	Limitations
Lowest friction entry point - works in your editor	Less agentic - reactive, not proactive
Wide IDE support (VS Code, JetBrains, Neovim...)	Single-file focus - limited cross-project awareness
Broad language coverage (TS, Python, Java...)	Model flexibility - tied to OpenAI models

Where it sits: The **gateway tool**, where most developers first experience AI coding.

Cursor

The AI-First IDE

Cursor - What It Is

- Not a plugin - a **standalone IDE** built on VS Code's foundation
- Designed from the ground up for **AI-first development**
- Where Copilot *adds* AI to your editor, Cursor *makes AI the central workflow*

Cursor - Three Interaction Layers

1. **Tab completion** - like Copilot, but context-aware across your *entire project*
2. **Inline editing (Cmd+K)** - select code, describe what you want, Cursor rewrites it
3. **Composer / Agent mode** - the big difference:
    - Reads and modifies **multiple files**
    - Creates new files and folders
    - Runs terminal commands
    - Iterates on errors autonomously

Cursor - Agent Mode

Copilot **suggests**.
Cursor can **act**.

Give it a task like "*add confetti effect when a chat finishes*" and it creates files, modifies files, updates configs, and checks for errors.

ACADEMY
- .cursor
- .next
- .vercel
- app
- components
- hooks
- lib
  - llm
  - confetti.ts U
  - course.ts
  - fs.ts
  - utils.ts
- logs
- node_modules
- public
- .env.local
- .gitignore
- bun.lock M
- components.json
- eslint.config.mjs
- mdx-components.tsx
- next-env.d.ts
- next.config.ts
- package.json M
- postcss.config.mjs
- README.md

OUTLINE
TIMELINE

TS confetti.ts U

lib > TS confetti.ts > ...

```typescript
import confetti from 'canvas-confetti';

/**
 * Triggers a confetti celebration when a chat finishes
 */
export function triggerChatConfetti() {
  // Create a nice confetti burst from the center
  const duration = 2000;
  const animationEnd = Date.now() + duration;
  const defaults = {
    startVelocity: 30,
    spread: 360,
    ticks: 60,
    zIndex: 1000
  };

  function randomInRange(min: number, max: number) {
    return Math.random() * (max - min) + min;
  }

  // Create multiple bursts for a more exciting effect
  const interval = setInterval(() => {
    const timeLeft = animationEnd - Date.now();

    if (timeLeft <= 0) {
      clearInterval(interval);
      return;
    }

    const particleCount = 50 * (timeLeft / duration);

    // Left side
    confetti({
      ...defaults,
```

1 / 1 Undo all ⇧⌘⌫ Keep all ⌘⏎ 1 / 2 files

Celebrate chat completion wit

Lets pop confetti when a chat finishes

Create a confetti comp... 1 of 5

TS ...ti.ts (new) +65

Now let me update the `useChatWithTokens` hook to trigger confetti when a chat finishes:

Add confetti trigger to ... 2 of 5

TS ...ithTokens.ts +1

TS ...ithTokens.ts +3

Perfect! Now let me check for any linting errors and test the implementation:

No linter errors found

Excellent! Now let me update the todos and test the confetti functionality:

4 of 5 Done View All

4 of 5 To-dos

2 Files Stop

@ 1 Tab bun run dev Terminal

Plan, search, build anything

⌘I claude-4-sonnet

15

Agents main* 0 0 Cursor Tab Ln 1, Col 1 Spaces: 4 UTF-8 LF TypeScript Prettier

Cursor - Multi-File Editing

Agent mode produces changes across your entire project, all in one step.

You review the diffs and accept or reject each change. This is the key workflow shift: you become the **reviewer**, not the typist.

Agent IDE

agents-ui

Add -E option for extended regex

feat-parallel-search-oswR6 by GPT 5

Create PR

Composer	GPT-5 Codex	Sonnet 4
2 files · +137-59	Planning next m...	1 file · +89-4

Add -E option for extended regex include updates to src/flags.rs, src/arg...

3 To-dos 2/3

All tests pass and the binary builds successfully. The -E option works as expected, allowing extended regular expressions (patterns like \d+, line|foo, etc.).

3 Files Edited Review

Cargo.toml workspace/searcher	+3
flags.rs workspace/searcher/src	+6-3
args.rs workspace/searcher/src	+7-2

3 Files Undo All Apply All

Plan, @ for contexts, / for commands

Agent Sonnet 4.5

Review: Add -E...

Pending 2 Files +12 -2 Undo All Apply All

args.rs workspace/searcher/src +7-2

```
2     pub enum Flag {
3 -       Help,
4         Version,
5 +   +   /// Enable extended regular expressions
6 +   +   ExtendedRegex,
7     }
      15 Lines hidden
23            .arg(clap::Arg::new("version").short('V').long("version"))
24 +  +       .arg(clap::Arg::new("extended-regex")
25 +  +           .short('E')
26 +  +           .long("extended-regex")
27 +  +           .help("Use extended regular expressions for the pattern"))
28    }
```

flags.rs workspace/searcher/src +6-3

```
11    pub struct SearchConfig {
12 -      pub use_regex: bool,
13        /// When true, interpret the pattern using *extended* regular expres
14 +      /// (set by `-E` / `--extended-regex`).
15 +      pub use_regex: bool,
16         pub verbose: bool,
17    }
18
```

Cursor - Strengths & Limitations

Strengths	Limitations
Agent mode - plan, execute, iterate across files	Standalone app - must switch from your current editor
Context-aware across your entire project	Agent mode has a learning curve (context, prompts)
Familiar VS Code interface - gentle onboarding	Pro plan needed for meaningful agent usage

Cursor - Our Primary Course Tool

Cursor is the tool you'll use most during this course:

- **This afternoon:** configure your Cursor environment
- **Tomorrow (Day 2):** deep dive into agent mode, context windows, MCP, system prompts
- **Days 2-5:** build CalcApp using Cursor as your daily driver

Claude Code

The Terminal Agent

Claude Code - What It Is

- Anthropic's **CLI-native coding agent**
- No IDE. No GUI. Just your terminal and a conversation.
- You describe a goal, Claude Code reads your codebase, plans, writes code, runs commands, and iterates.

A **fundamentally different interaction model**.

Claude Code - How You Use It

Open a terminal in your project and start a conversation:

```
$ claude

> Add a REST API endpoint for user registration
  with email validation and password hashing.
  Include tests.
```

Claude Code autonomously: reads your project → identifies patterns → creates/modifies files → writes tests → runs them → reports back. You describe *what*, it figures out *how*.

Claude Code - Task-Oriented Workflow

No ghost text.
No sidebar chat.

You give a **goal**.
The agent **executes**.
You **review** the result.

✳ Test Coverage ⌥⌘2

Claude Code v2.0.0
Sonnet 4.5 · Claude Enterprise
/Users/boris/code/demo

```
> audit and improve test coverage

· Grooving… (esc to interrupt)

>

  demo (demo)                                  Thinking on (tab to toggle)
```

Claude Code - What Makes It Unique

1. **No IDE lock-in** - works with any editor or none
2. **True agentic workflow** - reads, plans, writes, tests, iterates
3. **Terminal-native** - fits into git, npm, docker workflows
4. **Deep codebase understanding** - navigates thousands of files

Day 6: live demo of Claude Code tackling a real SE challenge.

Claude Code - Strengths & Limitations

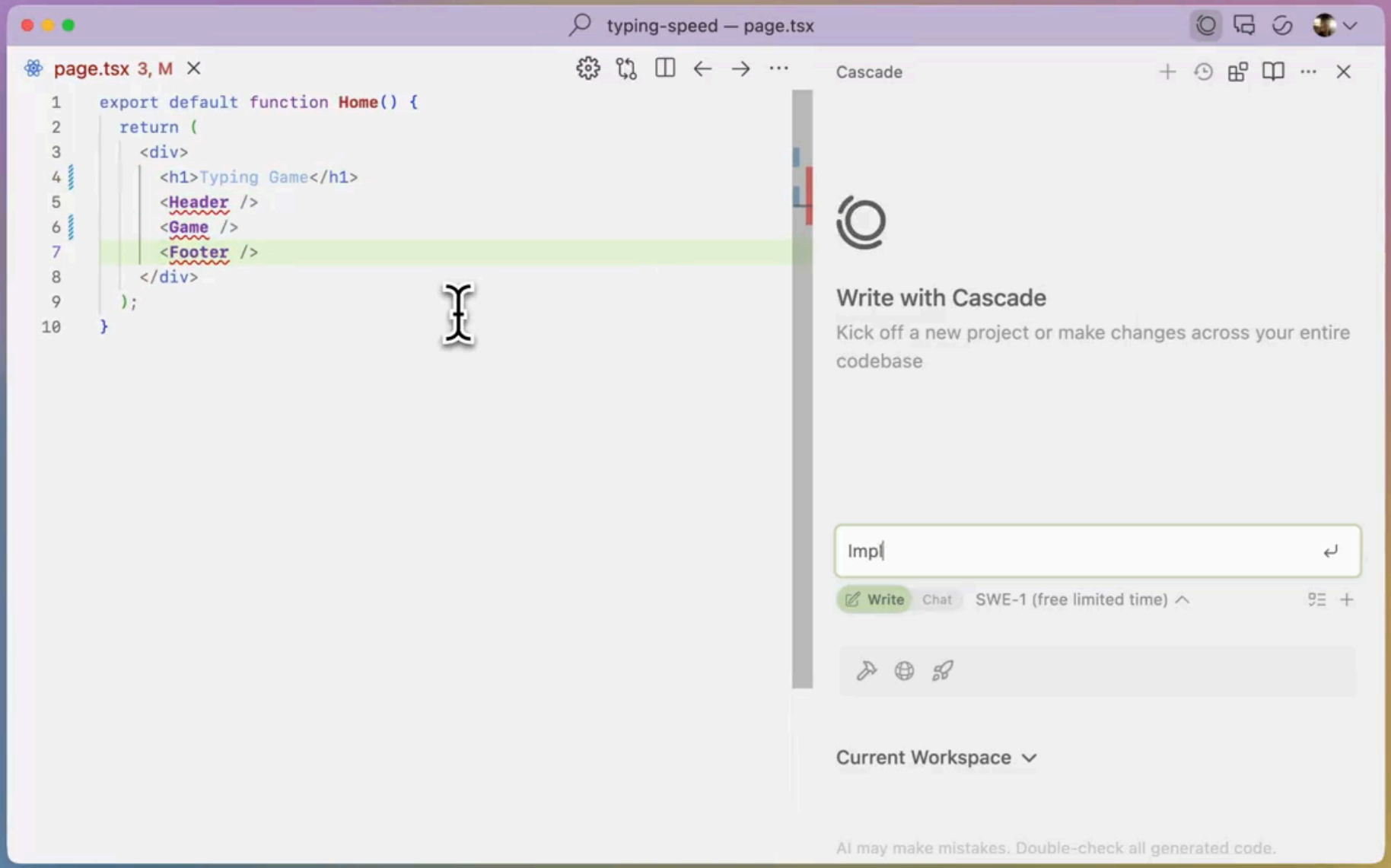
Strengths	Limitations
Highest autonomy - complex multi-file tasks	No real-time visual feedback in an IDE
Editor-agnostic - use any editor or none	Steeper learning curve (terminal-first)
Powerful for refactoring, migrations, cross-cutting changes	Requires trust + disciplined review habits

Beyond the Big Three

Windsurf

Position: AI-first IDE, similar to Cursor.

- By Codeium - Cursor's closest competitor
- Standout feature: **Cascade** - structured agentic workflows
- More opinionated than Cursor's open-ended agent mode


typing-speed — page.tsx
page.tsx 3, M
export default function Home() {
return (

<h1>Typing Game</h1>
<Header />
<Game />
<Footer />

);
}
Cascade
Write with Cascade
Kick off a new project or make changes across your entire codebase
Impl
Write
Chat
SWE-1 (free limited time)
Current Workspace
AI may make mistakes. Double-check all generated code.

OpenAI Codex

Position: Cloud-based autonomous agent, the far end of the spectrum.

- Submit a task, Codex works **asynchronously in the cloud**
- Come back later to review a pull request
- Like handing work to a developer who codes overnight

Trade-off: maximum autonomy, minimum control mid-flight.

New thread
Automations
Skills

Threads

Codex
- Create Codex app CTA 4h
- Implement dark mode 8h

ChatGPT
- Voice mode shortcuts 2h

Sora
- Persist prompt presets 5h

Atlas
- Add Status filter facet 3h

Create Codex app CTA openai/codex

Open | Commit

Create a compelling launch hero for the new Codex app on openai.com/codex

I'll update the hero copy to clearly communicate what Codex app does, add outcome-focused bullets, and ensure the CTAs align with launch goals.

Thought 7s

Explored 2 files

Edited hero.tsx ✓

Read build.py

Edited build.py ✓

Updated the launch hero to emphasize real developer outcomes (repo understanding, safe execution, PR delivery), and aligned the CTAs with launch intent.

Ask Codex anything

+ GPT-5.3-Codex

2 files changed +9 -6

src/hero.tsx +8 -5

```
  export const hero = {
-   eyebrow: "New",
-   title: "Codex",
-   subtitle: "AI for developers",
+   eyebrow: "Introducing",
+   title: "Codex app",
+   subtitle: "Your AI pair programmer",
+   primaryCta: "Get started",
+   secondaryCta: "Download the CLI",
  };

  export const heroBullets = [
-   "Write code faster",
-   "Understand any repo",
+   "Understands your repo in seconds",
+   "Executes commands safely in a sandbox",
+   "Turns issues into reviewed, production-ready PRs",
  ];
```

tools/build.py +1 -1

```
  def build():
-     print("building")
+     print("building launch hero…")

  if __name__ == "__main__":
     build()
```

Local LLMs

Position: Varies - autocomplete to light agent, depending on model.

- **Ollama**, **LM Studio** - run models on your own hardware
- Use case: **privacy** (code stays local) + **offline access**
- Trade-off: smaller, less capable models than cloud alternatives
- Great for autocomplete; struggle with complex multi-file tasks

```
codex --oss
ollama@ollamas-computer ~ % codex --oss

>_ OpenAI Codex (v0.85.0)

model:     gpt-oss:20b   /model to change
directory: ~

Tip: Use /mcp to list configured MCP tools.

› Write tests for @filename

100% context left · ? for shortcuts
```

Comparison Framework

Dimension	Copilot	Cursor	Claude Code	Windsurf	Codex	Local LLMs
Interaction	Inline + chat	IDE + agent	CLI + agent	IDE + agent	Cloud async	Inline / chat
Autonomy	Low	Med-High	High	Med-High	Very High	Low-Med
IDE dependency	Plugin	Standalone	None	Standalone	Web	Plugin / CLI
Model flexibility	Limited	High	Claude	Codeium	OpenAI	Full
Pricing	Free / Pro	Free / Pro	Free / Pro	Free / Pro	Pro / Team	Free

Key Insight

These tools are **not mutually exclusive**.

Many developers use:

- **Copilot** for quick completions
- **Cursor** or **Claude Code** for bigger tasks

The spectrum isn't about picking one tool - it's about knowing **when to reach for which one**.

Checkpoint

Which tool would you reach for first?

Think about what you've just seen:

- GitHub Copilot, Cursor, Claude Code
- Windsurf, OpenAI Codex, Local LLMs

Which one appeals to you most, and why?

Where Is This Going?

Future Trends

1. **Agent convergence** - every tool is racing toward more agency. The "assistant vs agent" line is blurring fast.
2. **Local models catching up** - open-weight models improve rapidly. Today's cloud-only capabilities become tomorrow's local capabilities.
3. **Tool specialization** - as the baseline rises, differentiation comes from deeper integration, better context management, and unique workflows.

What's Next

When	What
Today, 1:30 PM	Install Cursor and configure your environment
Today, 2:30 PM	Build FreeSelfApp - your first AI-assisted prototype
Tomorrow, 9:15 AM	Deep dive: Cursor agent mode, MCP, context management
Day 6, 10:30 AM	Live demo: Claude Code in a real SE workflow

Questions?

Tools Ecosystem - Day 1

Tiago Sousa